

CSE 3204
Formal Language, Automata and Computability



Lecture 14
Languages of a Pushdown Automata

Languages of a PDA

- Acceptance by Final State
 - A PDA accepts its input by consuming it and entering an accepting state.
- Acceptance by Empty State
 - The set of strings that cause the PDA to empty its stack starting from the initial ID

6.2.1 Acceptance by Final State

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a PDA. Then $L(P)$, the *language accepted by P by final state*, is

$$\{w \mid (q_0, w, Z_0) \vdash_P^* (q, \epsilon, \alpha)\}$$

for some state q in F and any stack string α . That is, starting in the initial ID with w waiting on the input, P consumes w from the input and enters an accepting state. The contents of the stack at that time is irrelevant.

The PDA P of Example 6.2 accepts string x by final state if and only if x is of the form $w.w^R$

(If) This part is easy; we have only to show the accepting computation of P . If $x = ww^R$, then observe that

$$(q_0, ww^R, Z_0) \vdash^* (q_0, w^R, w^R Z_0) \vdash (q_1, w^R, w^R Z_0) \vdash^* (q_1, \epsilon, Z_0) \vdash (q_2, \epsilon, Z_0)$$

(Only-if) This part is harder. First, observe that the only way to enter accepting state q_2 is to be in state q_1 and have Z_0 at the top of the stack. Also, any accepting computation of P will start in state q_0 , make one transition to q_1 , and never return to q_0 . Thus, it is sufficient to find the conditions on x such that $(q_0, x, Z_0) \vdash^* (q_1, \epsilon, Z_0)$; these will be exactly the strings x that P accepts by final state. We shall show by induction on $|x|$ the slightly more general statement:

- If $(q_0, x, \alpha) \vdash^* (q_1, \epsilon, \alpha)$, then x is of the form ww^R .

BASIS: If $x = \epsilon$, then x is of the form ww^R (with $w = \epsilon$). Thus, the conclusion is true, so the statement is true. Note we do not have to argue that the hypothesis $(q_0, \epsilon, \alpha) \vdash^* (q_1, \epsilon, \alpha)$ is true, although it is.

INDUCTION: Suppose $x = a_1 a_2 \cdots a_n$ for some $n > 0$. There are two moves that P can make from ID (q_0, x, α) :

1. $(q_0, x, \alpha) \vdash (q_1, x, \alpha)$. Now P can only pop the stack when it is in state q_1 . P must pop the stack with every input symbol it reads, and $|x| > 0$. Thus, if $(q_1, x, \alpha) \vdash^* (q_1, \epsilon, \beta)$, then β will be shorter than α and cannot be equal to α .
2. $(q_0, a_1 a_2 \cdots a_n, \alpha) \vdash (q_0, a_2 \cdots a_n, a_1 \alpha)$. Now the only way a sequence of moves can end in (q_1, ϵ, α) is if the last move is a pop:

$$(q_1, a_n, a_1 \alpha) \vdash (q_1, \epsilon, \alpha)$$

In that case, it must be that $a_1 = a_n$. We also know that

$$(q_0, a_2 \cdots a_n, a_1 \alpha) \vdash^* (q_1, a_n, a_1 \alpha)$$

By Theorem 6.6, we can remove the symbol a_n from the end of the input, since it is not used. Thus,

$$(q_0, a_2 \cdots a_{n-1}, a_1 \alpha) \vdash^* (q_1, \epsilon, a_1 \alpha)$$

Since the input for this sequence is shorter than n , we may apply the inductive hypothesis and conclude that $a_2 \cdots a_{n-1}$ is of the form yy^R for some y . Since $x = a_1 yy^R a_n$, and we know $a_1 = a_n$, we conclude that x is of the form ww^R ; specifically $w = a_1 y$.

6.2.2 Acceptance by Empty Stack

For each PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, we also define

$$N(P) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \epsilon, \epsilon)\}$$

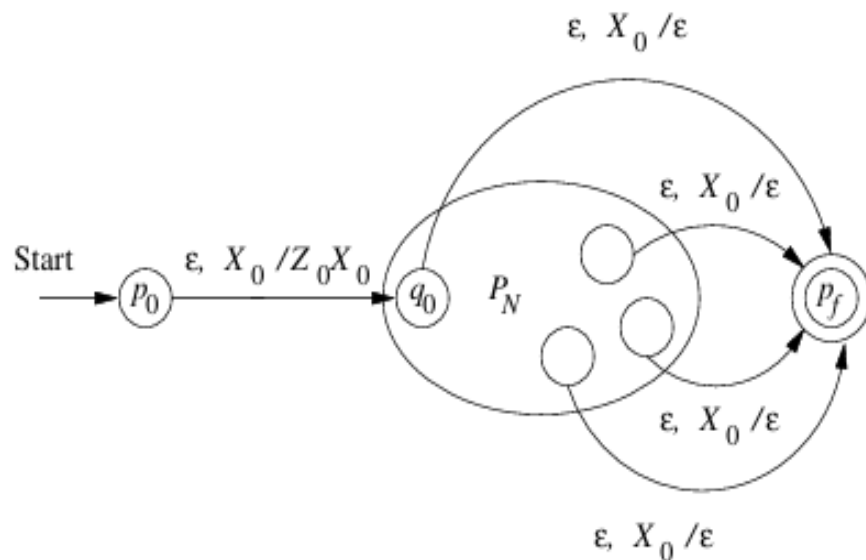
for any state q . That is, $N(P)$ is the set of inputs w that P can consume and at the same time empty its stack.²

Example 6.8: The PDA P of Example 6.2 never empties its stack, so $N(P) = \emptyset$. However, a small modification will allow P to accept L_{wwr} by empty stack as well as by final state. Instead of the transition $\delta(q_1, \epsilon, Z_0) = \{(q_2, Z_0)\}$, use $\delta(q_1, \epsilon, Z_0) = \{(q_2, \epsilon)\}$. Now, P pops the last symbol off its stack as it accepts, and $L(P) = N(P) = L_{wwr}$. $\square$

From Empty Stack to Final State

Theorem 6.9: If $L = N(P_N)$ for some PDA $P_N = (Q, \Sigma, \Gamma, \delta_N, q_0, Z_0)$, then there is a PDA P_F such that $L = L(P_F)$.

PROOF: The idea behind the proof is in Fig. 6.4. We use a new symbol X_0 , which must not be a symbol of Γ ; X_0 is both the start symbol of P_F and a marker on the bottom of the stack that lets us know when P_N has reached an empty stack. That is, if P_F sees X_0 on top of its stack, then it knows that P_N would empty its stack on the same input.



The specification of P_F is as follows:

$$P_F = (Q \cup \{p_0, p_f\}, \Sigma, \Gamma \cup \{X_0\}, \delta_F, p_0, X_0, \{p_f\})$$

where δ_F is defined by:

1. $\delta_F(p_0, \epsilon, X_0) = \{(q_0, Z_0 X_0)\}$. In its start state, P_F makes a spontaneous transition to the start state of P_N , pushing its start symbol Z_0 onto the stack.
2. For all states q in Q , inputs a in Σ or $a = \epsilon$, and stack symbols Y in Γ , $\delta_F(q, a, Y)$ contains all the pairs in $\delta_N(q, a, Y)$.
3. In addition to rule (2), $\delta_F(q, \epsilon, X_0)$ contains (p_f, ϵ) for every state q in Q .

We must show that w is in $L(P_F)$ if and only if w is in $N(P_N)$.

Figure 6.4: P_F simulates P_N and accepts if P_N empties its stack

From Empty Stack to Final State

We must show that w is in $L(P_F)$ if and only if w is in $N(P_N)$.

(If) We are given that $(q_0, w, Z_0) \vdash_{P_N}^* (q, \epsilon, \epsilon)$ for some state q . Theorem 6.5 lets us insert X_0 at the bottom of the stack and conclude $(q_0, w, Z_0 X_0) \vdash_{P_N}^* (q, \epsilon, X_0)$. Since by rule (2) above, P_F has all the moves of P_N , we may also conclude that $(q_0, w, Z_0 X_0) \vdash_{P_F}^* (q, \epsilon, X_0)$. If we put this sequence of moves together with the initial and final moves from rules (1) and (3) above, we get:

$$(p_0, w, X_0) \vdash_{P_F} (q_0, w, Z_0 X_0) \vdash_{P_F}^* (q, \epsilon, X_0) \vdash_{P_F} (p_f, \epsilon, \epsilon) \quad (6.1)$$

Thus, P_F accepts w by final state.

Only-if part:

Thus, any computation of P_F that accepts w must look like sequence (6.1). Moreover, the middle of the computation — all but the first and last steps — must also be a computation of P_N with X_0 below the stack. The reason is that, except for the first and last steps, P_F cannot use any transition that is not also a transition of P_N , and X_0 cannot be exposed or the computation would end at the next step. We conclude that $(q_0, w, Z_0) \vdash_{P_N}^* (q, \epsilon, \epsilon)$. That is, w is in $N(P_N)$.

□

From Empty Stack to Final State

Example 6.10: Let us design a PDA that processes sequences of if's and else's in a C program, where i stands for if and e stands for else. Recall from Section 5.3.1 that there is a problem whenever the number of else's in any prefix exceeds the number of if's, because then we cannot match each else against its previous if. Thus, we shall use a stack symbol Z to count the difference between the number of i 's seen so far and the number of e 's. This simple, one-state PDA, is suggested by the transition diagram of Fig. 6.5.

We shall push another Z whenever we see an i and pop a Z whenever we see an e . Since we start with one Z on the stack, we actually follow the rule that if the stack is Z^n , then there have been $n - 1$ more i 's than e 's. In particular, if the stack is empty, then we have seen one more e than i , and the input read so far has just become illegal for the first time. It is these strings that our PDA accepts by empty stack. The formal specification of P_N is:

$$P_N = (\{q\}, \{i, e\}, \{Z\}, \delta_N, q, Z)$$

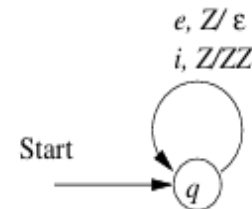


Figure 6.5: A PDA that accepts the if/else errors by empty stack

where δ_N is defined by:

1. $\delta_N(q, i, Z) = \{(q, ZZ)\}$. This rule pushes a Z when we see an i .
2. $\delta_N(q, e, Z) = \{(q, \epsilon)\}$. This rule pops a Z when we see an e .

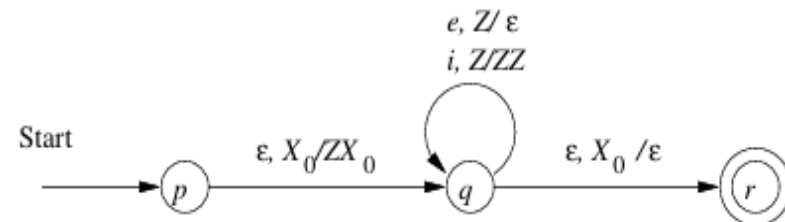


Figure 6.6: Construction of a PDA accepting by final state from the PDA of Fig. 6.5

From Empty Stack to Final State

$$P_F = (\{p, q, r\}, \{i, e\}, \{Z, X_0\}, \delta_F, p, X_0, \{r\})$$

where δ_F consists of:

1. $\delta_F(p, \epsilon, X_0) = \{(q, ZX_0)\}$. This rule starts P_F simulating P_N , with X_0 as a bottom-of-stack-marker.
2. $\delta_F(q, i, Z) = \{(q, ZZZ)\}$. This rule pushes a Z when we see an i ; it simulates P_N .
3. $\delta_F(q, e, Z) = \{(q, \epsilon)\}$. This rule pops a Z when we see an e ; it also simulates P_N .
4. $\delta_F(q, \epsilon, X_0) = \{(r, \epsilon)\}$. That is, P_F accepts when the simulated P_N would have emptied its stack.

□

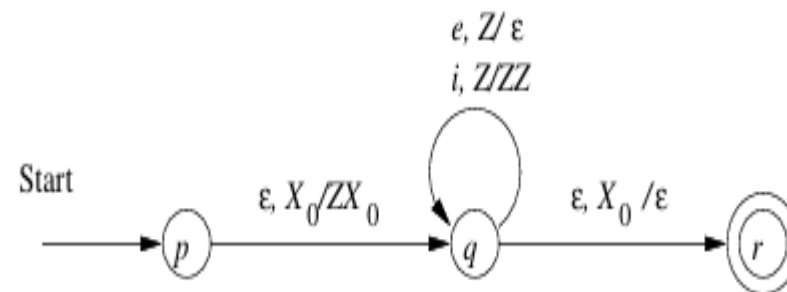


Figure 6.6: Construction of a PDA accepting by final state from the PDA of Fig. 6.5

From Final State to Empty State

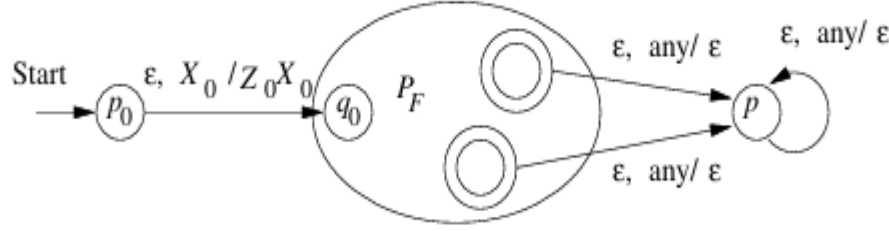


Figure 6.7: P_N simulates P_F and empties its stack when and only when P_N enters an accepting state

Theorem 6.11: Let L be $L(P_F)$ for some PDA $P_F = (Q, \Sigma, \Gamma, \delta_F, q_0, Z_0, F)$. Then there is a PDA P_N such that $L = N(P_N)$.

PROOF: The construction is as suggested in Fig. 6.7. Let

$$P_N = (Q \cup \{p_0, p\}, \Sigma, \Gamma \cup \{X_0\}, \delta_N, p_0, X_0)$$

where δ_N is defined by:

1. $\delta_N(p_0, \epsilon, X_0) = \{(q_0, Z_0 X_0)\}$. We start by pushing the start symbol of P_F onto the stack and going to the start state of P_F .
2. For all states q in Q , input symbols a in Σ or $a = \epsilon$, and Y in Γ , $\delta_N(q, a, Y)$ contains every pair that is in $\delta_F(q, a, Y)$. That is, P_N simulates P_F .

3. For all accepting states q in F and stack symbols Y in Γ or $Y = X_0$, $\delta_N(q, \epsilon, Y)$ contains (p, ϵ) . By this rule, whenever P_F accepts, P_N can start emptying its stack without consuming any more input.
4. For all stack symbols Y in Γ or $Y = X_0$, $\delta_N(p, \epsilon, Y) = \{(p, \epsilon)\}$. Once in state p , which only occurs when P_F has accepted, P_N pops every symbol on its stack, until the stack is empty. No further input is consumed.

Now, we must prove that w is in $N(P_N)$ if and only if w is in $L(P_F)$.

(If) Suppose $(q_0, w, Z_0) \xrightarrow{P_F}^* (q, \epsilon, \alpha)$ for some accepting state q and stack string α . Using the fact that every transition of P_F is a move of P_N , and invoking Theorem 6.5 to allow us to keep X_0 below the symbols of Γ on the stack, we know that $(q_0, w, Z_0 X_0) \xrightarrow{P_N}^* (q, \epsilon, \alpha X_0)$. Then P_N can do the following:

$$(p_0, w, X_0) \xrightarrow{P_N} (q_0, w, Z_0 X_0) \xrightarrow{P_N}^* (q, \epsilon, \alpha X_0) \xrightarrow{P_N}^* (p, \epsilon, \epsilon)$$

The first move is by rule (1) of the construction of P_N , while the last sequence of moves is by rules (3) and (4). Thus, w is accepted by P_N , by empty stack.

From Final State to Empty State

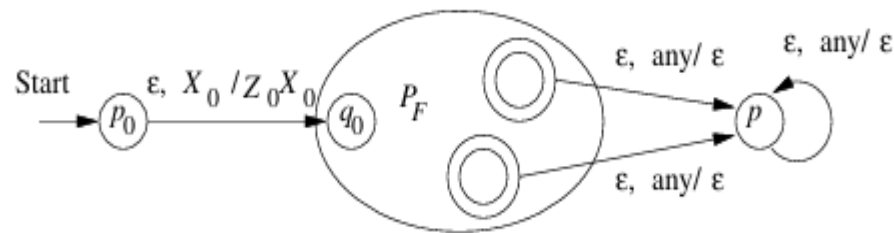


Figure 6.7: P_N simulates P_F and empties its stack when and only when P_N enters an accepting state

(Only-if) The only way P_N can empty its stack is by entering state p , since X_0 is sitting at the bottom of stack and X_0 is not a symbol on which P_F has any moves. The only way P_N can enter state p is if the simulated P_F enters an accepting state. The first move of P_N is surely the move given in rule (1). Thus, every accepting computation of P_N looks like

$$(p_0, w, X_0) \vdash_{P_N} (q_0, w, Z_0 X_0) \stackrel{*}{\vdash}_{P_N} (q, \epsilon, \alpha X_0) \stackrel{*}{\vdash}_{P_N} (p, \epsilon, \epsilon)$$

where q is an accepting state of P_F .

Moreover, between ID's $(q_0, w, Z_0 X_0)$ and $(q, \epsilon, \alpha X_0)$, all the moves are moves of P_F . In particular, X_0 was never the top stack symbol prior to reaching ID $(q, \epsilon, \alpha X_0)$.⁴ Thus, we conclude that the same computation can occur in P_F , without the X_0 on the stack; that is, $(q_0, w, Z_0) \stackrel{*}{\vdash}_{P_F} (q, \epsilon, \alpha)$. Now we see that P_F accepts w by final state, so w is in $L(P_F)$. $\square$

- Section 6.2 of Chapter 6